ChatGPT

Problemi e Soluzioni nella Gestione delle Prenotazioni (WhatsApp AI)

La conversazione di test evidenzia diversi **bug** nel flusso di modifica delle prenotazioni via chatbot. Di seguito analizziamo ciascun problema, ne individuiamo le cause probabili e proponiamo soluzioni dettagliate. Queste indicazioni possono servire come guida, anche per un assistente AI, per correggere il codice.

Problema 1: La prenotazione vecchia non viene cancellata

Descrizione: Dopo aver modificato la data/orario di una prenotazione, la prenotazione originale rimane ancora nella lista. Nell'esempio, l'utente sposta la prenotazione del 12/12 ore 09:00, ma questa continua a comparire nell'elenco insieme alla nuova prenotazione (12/12 ore 15:30) invece di essere aggiornata o rimossa.

Cause possibili:

- *Duplice creazione invece di aggiornamento:* Il sistema ha **aggiunto** una nuova prenotazione senza eliminare o aggiornare quella esistente. Dalla conversazione si nota il messaggio di conferma " *Prenotato: … il 12/12 alle 15:30.*", tipico di una nuova prenotazione, anziché un messaggio di aggiornamento [1]. Questo suggerisce che la logica di modifica abbia inavvertitamente eseguito il flusso di **creazione** di un nuovo evento invece di aggiornare l'evento esistente.

- *Mancata cancellazione dell'evento originale:* Nel codice, la funzione `_editApplyUpdate` è pensata per applicare le modifiche alle prenotazioni esistenti (con possibilità di cancellare e ricreare l'evento) [2] [3]. Se però questo percorso non viene eseguito (ad esempio a causa di un errore o di un flusso alternativo), l'evento originale resta intatto. In particolare, se la modifica della data bypassa `_editApplyUpdate` e utilizza invece `persistBooking` (inserendo un nuovo record) senza chiamare `markBookingCancelled` sul vecchio, si ottiene un duplicato.

Soluzione:
Bisogna assicurarsi che modificare una prenotazione **non crei duplicati**. Ecco le azioni consigliate:

- **Usare `_editApplyUpdate`:** Dopo che l'utente sceglie la nuova data/orario, chiamare `_editApplyUpdate` per aggiornare l'evento esistente invece di crearne uno nuovo. Questa funzione gestisce sia l'aggiornamento su Google Calendar (tramite `updateEventSafe`) sia il salvataggio su `bookings.json` [4] [5]. In caso di cambio data, si può utilizzare il parametro `newStartISO` (e `forceRecreate=true` se necessario) in modo che `_editApplyUpdate` cancelli l'evento vecchio e ne crei uno nuovo con lo stesso ID di prenotazione [2] [3]. In questo modo il record nel file viene sovrascritto invece di duplicarsi.

- **Annullare esplicitamente l'originale (fallback):** Se per qualche ragione si è scelto di creare un nuovo evento invece di aggiornare, allora subito dopo bisogna marcare come cancellato l'evento precedente. Si può usare `markBookingCancelled( eventIdOriginale )` o una patch sul

record (`status: "cancelled"`) [6] [7]. Ciò rimuoverà la vecchia prenotazione dalla lista delle attive. Tuttavia, questa è una soluzione meno elegante – è preferibile aggiornare in-place.

- **Confermare l'aggiornamento all'utente:** In caso di successo, inviare un messaggio tipo "*Prenotazione aggiornata*" con i nuovi dettagli, utilizzando la funzione `showEditMenu(updated)` per mostrare le opzioni successive [1]. Questo feedback distingue chiaramente l'aggiornamento dalla creazione di una nuova prenotazione.

Implementando queste correzioni, quando l'utente modifica una prenotazione esistente, il sistema **aggiornerà il record esistente** (o cancellerà quello vecchio) invece di mantenerne due. Ciò risolve il doppione nell'elenco delle prenotazioni future.

Problema 2: Persistenza del bug "Quel posto è appena stato occupato . Disponibili invece: …"

Descrizione: Il messaggio di errore *"Quel posto è appena stato occupato . Disponibili invece: …"* compare in modo inaspettato durante il cambio data [8]. Nel flusso previsto, questo messaggio dovrebbe apparire solo se, dopo aver selezionato un orario, qualcun altro ha prenotato contemporaneamente quello slot. Nel nostro caso, però, è comparso subito dopo la scelta della nuova **data** ("domani"), prima ancora che l'utente selezionasse un orario, elencando orari alternativi del giorno sbagliato (vedi Problema 4). Ciò indica un comportamento indesiderato o prematuro del controllo di disponibilità.

Cause possibili:

- *Controllo di disponibilità sullo slot originale:* L'ipotesi è che il sistema abbia tentato immediatamente di ri-prenotare lo **stesso orario** sulla nuova data e abbia rilevato un conflitto. Probabilmente, dopo che l'utente ha inserito "domani", il codice ha provato a mantenere l'ora originale (es. 09:00) sul nuovo giorno. Se quello slot risultava occupato (o se il controllo è stato fatto sul giorno sbagliato – vedi Problema 4), il sistema ha subito attivato il messaggio di conflitto. In pratica, il bot ha saltato la fase di proposta orari liberi, tentando un aggiornamento automatico.

- *Conflitto con la stessa prenotazione (falso positivo):* Un bug sottostante potrebbe essere che il sistema non escluda la **prenotazione originale** nel controllo di conflitto. Nel caso in cui il controllo venga eseguito sul vecchio giorno/ora per errore, trova certamente "occupato" perché c'è già l'evento dell'utente stesso a quell'ora. La funzione `slotOverlapsExistingBookings` segna lo slot come occupato se si sovrappone a qualsiasi evento esistente [9], senza ignorare l'evento corrente in fase di spostamento. Questo genera un falso conflitto.

Soluzione:
Bisogna gestire meglio la logica di controllo disponibilità quando si cambia data:

- **Usare la data corretta per il controllo (vedi Problema 4):** Assicurarsi che il check avvenga sul giorno scelto dall'utente e non sul giorno originale. Dopo che l'utente inserisce una nuova data, aggiornare subito la variabile dello stato che rappresenta il "giorno desiderato" (es. `col.when`) a questa nuova data prima di qualsiasi controllo. In questo modo, la funzione di hold o di fetch userà il giorno giusto.

- **Tentare la stessa ora sul nuovo giorno (opzionale):** È una buona idea provare a mantenere l'ora originale sul nuovo giorno per comodità dell'utente, ma va fatto in modo sicuro. Si può implementare così:

```
// pseudo-codice
const newStart = new Date(newDay);
newStart.setHours(origHour, origMin);
if (slotDisponibile(newStart, professional)) {
    // procedi con update immediato
} else {
    // slot occupato, passa a lista orari disponibili
}
```

  Dove `slotDisponibile` utilizza un controllo di conflitto simile a `acquireSoftHold` ma **ignorando** l'evento corrente. In alternativa, si può chiamare direttamente Google Calendar per provare l'update: se fallisce per conflitto, si cattura l'errore.

- **Escludere l'evento originale dal conflitto:** Modificare `slotOverlapsExistingBookings` o la logica di hold per ignorare il booking in corso di modifica. Ad esempio, si potrebbe passare l'`eventId` da escludere, e filtrarlo dalle occupazioni prima di determinare `holdOk`. In mancanza di ciò, una soluzione semplice è *marcare temporaneamente* la prenotazione come cancellata prima di verificare il nuovo slot, oppure rimuoverla dalla lista caricata in memoria durante il controllo.

- **Mostrare il messaggio appropriato:** Se lo slot è veramente occupato da **altri**, il messaggio "Quel posto è appena stato occupato …" con l'elenco alternative è corretto. Tuttavia, non deve triggerare prematuramente. Dopo le modifiche sopra, questo messaggio apparirà solo dopo che l'utente avrà effettivamente scelto un orario che nel frattempo è stato preso (evento poco frequente).

Implementando queste correzioni, il bot gestirà meglio il cambio data: tenterà l'ora originale sul nuovo giorno e, solo se necessario, mostrerà gli orari alternativi. Ciò ridurrà la comparsa non necessaria del messaggio di conflitto. In sintesi, **il bug persisteva perché il sistema controllava lo slot sbagliato o non filtrava l'evento corrente** – correggendo questi aspetti, l'errore "Quel posto è appena stato occupato" verrà mostrato solo quando realmente applicabile.

Problema 3: Errore `ReferenceError: normalizeSlotsDetail is not defined`

Descrizione: Nei log compare un errore runtime che interrompe l'esecuzione: `ReferenceError: normalizeSlotsDetail is not defined` [10]. Questo avviene durante la gestione del messaggio utente, precisamente quando il codice tenta di chiamare la funzione `normalizeSlotsDetail(...)`. L'errore indica che la funzione non è stata definita da nessuna parte prima del suo utilizzo.

Cause possibili:

- *Funzione mancante o non importata:* È probabile che `normalizeSlotsDetail` fosse una funzione utile a formattare o filtrare la lista di slot orari (forse per unire slot contigui o allegare

dettagli). In alcuni punti del codice l'uso di `normalizeSlotsDetail` è protetto da un controllo (`typeof normalizeSlotsDetail === "function" ? ... : ...`), ma in almeno una chiamata no [11] [12]. Se la funzione non è definita, JavaScript lancia un ReferenceError appena si prova a invocarla. Ciò potrebbe essere dovuto a una dimenticanza: magari la funzione era definita in una versione precedente o doveva essere importata ma non lo è stata.

- *Implementazione incompleta:* Visto che esistono riferimenti multipli a `normalizeSlotsDetail`, è possibile che l'implementazione fosse prevista ma è andata persa. Ad esempio, se originariamente `fetchSlots` restituiva un formato grezzo (lista di stringhe orari o una mappa) e `normalizeSlotsDetail` doveva trasformarlo in un array di oggetti `{time, pros}` o simili, l'assenza di tale funzione rompe il flusso. Nel nostro caso, la chiamata avviene quando si recuperano gli slot disponibili per il cambio appuntamento [12].

Soluzione:
Bisogna definire o rimuovere questa funzione per eliminare l'errore:

- **Implementare** `normalizeSlotsDetail`: Se si conosce lo scopo previsto, conviene scriverne una versione anche minima. Ad esempio, se `fetchSlots({ wantDetail:true })` già restituisce array di oggetti con chiavi `time` e `pros`, una possibile implementazione è un **pass-through** che verifica/normalizza questi oggetti. Esempio:

```
function normalizeSlotsDetail(list) {
    if (!Array.isArray(list)) return [];
    return list.map(slot => {
        // Assicura proprietà consistenti
        if (typeof slot === 'string') {
            return { time: slot, pros: [] };
        }
        return {
            time: slot.time || slot.start || "",
            pros: slot.pros || slot.professionals || []
        };
    });
}
```

Questo garantisce che ogni elemento abbia almeno `time` (stringa "HH:MM") e una lista `pros` (professionisti disponibili in quello slot). Si può adattare in base alla struttura esatta ritornata da `fetchSlots`. L'importante è che la funzione **esista** e ritorni un array compatibile con gli usi successivi (ad es. `renderSlotsList` o mapping delle opzioni).

- **Rimuovere le chiamate o usare il fallback:** In alternativa, se `fetchSlots` già restituisce i dati nel formato desiderato, si può semplicemente **eliminare** l'uso di `normalizeSlotsDetail`. Infatti, in vari punti il codice prevede il fallback `: list`. Si può dunque modificare le chiamate non protette in modo simile:

```
const slots = typeof normalizeSlotsDetail === "function"
                ? normalizeSlotsDetail(await fetchSlots(...))
                : await fetchSlots(...);
```

oppure, più semplicemente, definire all'inizio `const normalizeSlotsDetail = list => list;` come funzione identità. Quest'ultima soluzione risolve immediatamente l'errore senza impattare la logica.

- **Verificare l'effetto sul rendering:** Una volta risolto l'errore, assicurarsi che la lista di orari venga mostrata correttamente. Se ad esempio prima venivano duplicati orari o visualizzati in modo strano, controllare se la normalizzazione serviva a unirli. In caso, implementare quella logica (ad es. unire slot contigui in un range tipo "10:00–12:00" se desiderato).

Con questa correzione, l'errore di riferimento sarà eliminato. Il codice potrà eseguire il blocco interessato (come il caricamento degli orari disponibili) senza interrompersi, permettendo al chatbot di proseguire la conversazione normalmente. In sintesi, **o si implementa la funzione mancante, o si tolgono i riferimenti ad essa**, garantendo che l'elaborazione continui.

Problema 4: Orari proposti sul giorno sbagliato durante il cambio data

Descrizione: Quando l'utente prova a cambiare la data di una prenotazione e lo slot originale sul nuovo giorno non è disponibile, il bot mostra una lista di orari alternativi riferiti al **giorno sbagliato** (il giorno originario invece del nuovo). Nell'esempio, dopo aver scelto "domani" come nuova data per l'appuntamento del 12/12, il sistema ha elencato orari che sembrano appartenere al 12/12 (10:00, 10:30, … 18:30 con Diana) invece che al giorno successivo. Questo ha confuso l'utente e portato a prenotare un orario sempre sul 12/12, aggiungendo un doppio appuntamento.

Cause possibili:

- *Mancato aggiornamento dello stato con la nuova data:* Il codice non ha sostituito il parametro di data corrente con la nuova data prima di generare gli slot disponibili. In particolare, sembra che la lista di orari alternativi sia stata ottenuta usando ancora `col.when` settato al giorno originale (12/12) [13] [14]. Se `col.when` non viene aggiornato a "26/11" (domani, rispetto al 25/11) al momento giusto, la chiamata `fetchSlots` fornisce le disponibilità per il giorno sbagliato.

- *Riutilizzo improprio del flusso "pick_time":* Come dedotto nel Problema 2, il bot potrebbe aver deviato sul flusso normale di scelta orario (`state.step = "pick_time"`) senza cambiare correttamente il contesto. In tal caso, `state.collected.when` è rimasto al vecchio valore. Il blocco di codice che aggiorna la lista orari in caso di conflitto (mostrando "Disponibili invece:") utilizza `col.when` per determinare il giorno [13]. Se questo valore è sbagliato, anche gli slot lo saranno.

Soluzione:
Bisogna assicurarsi che la **nuova data** scelta dall'utente sia usata ovunque correttamente prima di calcolare gli orari disponibili:

- **Aggiornare `col.when` subito dopo il parsing della data:** Nella gestione di `edit_reschedule_date`, dopo aver calcolato `newDay` dal messaggio utente, impostare ad esempio:

```
col.when = newDay;
state.edit.day = newDay;
```

(oltre a salvare lo state). Nel codice fornito, vediamo che `state.edit.day` è impostato, ma la struttura `col` non viene aggiornata [15]. Aggiungendo questa assegnazione, le funzioni generiche che si appoggiano a `col.when` useranno il giorno corretto.

- **Elencare gli orari sul nuovo giorno in modalità "modifica":** Idealmente, bisognerebbe seguire il flusso di modifica già predisposto. Il codice già prevede di chiamare `fetchSlots` con il `newDay` e mostrare *"Orari disponibili per [nuova data]..."* prima di passare allo step successivo [16] [17]. Dopo aver risolto i bug precedenti (soprattutto l'errore `normalizeSlotsDetail`), questo passaggio dovrebbe funzionare. Quindi, **non deviare sul flusso standard di prenotazione** ma restare nel flusso di edit: l'utente dovrebbe vedere ad esempio *"Orari disponibili per 26/11 (tengo il professionista attuale): ..."*.

- **Verificare i dati del nuovo slot prima di aggiornare:** Quando l'utente sceglie uno degli orari proposti, assicurarsi che il codice combini **il nuovo giorno con l'orario selezionato**. Nel blocco `edit_reschedule_time` vediamo che viene fatto correttamente usando `state.edit.day` (nuovo giorno) più l'ora scelta [18] [19]. Questo produce `newStartISO` per `_editApplyUpdate`. Bisogna però arrivarci col contesto giusto (ovvero senza essere finiti nel flusso `pick_time` globale per errore).

- **Pulizia dello stato in caso di fallback:** Dopo aver mostrato gli orari e ricevuto la scelta, il sistema aggiorna la prenotazione e torna al menu di modifica [20] [21]. Conviene verificare che, in caso di conflitti gestiti nel flusso normale, lo stato venga riportato correttamente nel flusso di modifica (ad esempio impostando `state.step = "edit_menu"` dopo l'aggiornamento, come nel codice, per tornare al menu corretto [21]).

Apportando questi cambiamenti, quando lo slot originale sul nuovo giorno non è libero, l'utente riceverà una lista di orari **pertinenti al giorno desiderato**, evitando confusione. Nel caso d'esempio, avrebbe visto gli orari disponibili per il 26/11 anziché per il 12/12, e la nuova prenotazione sarebbe stata fissata sul giorno giusto. In breve, **la chiave è aggiornare lo stato con la nuova data prima di calcolare le disponibilità**, mantenendo il flusso di modifica nel suo contesto.

Guida riassuntiva per l'AI (Risoluzione step-by-step):

1. **Eliminare i duplicati di prenotazione:** Modificare la logica di cambio appuntamento per aggiornare o cancellare la prenotazione originale. Utilizzare `_editApplyUpdate` con `newStartISO` per spostare l'orario; in caso di creazione di nuovo evento, chiamare `markBookingCancelled` sul vecchio. Verificare che dopo la modifica `getUserActiveBookingsAll` non ritorni doppioni (solo la prenotazione aggiornata deve risultare attiva).

2. **Correggere il trigger del messaggio "posto occupato":** Implementare il tentativo di mantenere l'ora originale sul nuovo giorno, ma assicurarsi di usare la nuova data e di escludere l'evento corrente dal controllo di conflitto. Aggiornare `col.when` prima di qualsiasi verifica. Mostrare "Disponibili invece:" solo se realmente lo slot sul nuovo giorno è già prenotato da qualcun altro.

3. **Definire `normalizeSlotsDetail` o rimuovere il suo utilizzo:** Aggiungere la definizione mancante (anche come semplice passthrough) in cima al file **util.js**, oppure racchiudere tutte le sue chiamate in controlli `typeof` per usare direttamente i dati grezzi. Assicurarsi che la formattazione degli slot (campo `time`, nomi professionisti in `pros`) sia coerente con quanto atteso dalle funzioni che costruiscono i messaggi al cliente [22].

4. **Utilizzare la nuova data per gli slot disponibili:** Quando l'utente fornisce una nuova data, impostare subito lo stato interno a tale data. Quindi chiamare `fetchSlots` per quel giorno e presentare gli orari disponibili correttamente. Nel caso di conflitto con l'ora originale, assicurarsi che la lista alternativa riguardi il giorno scelto (e non il precedente). Testare il caso in cui uno slot non sia disponibile per verificare che ora il bot dica, ad esempio, *"Disponibili invece [orari] per il giorno X"* in modo appropriato.

Seguendo questa guida, l'assistente AI (o lo sviluppatore) potrà correggere il flusso di modifica delle prenotazioni. Dopo le modifiche, il sistema dovrebbe permettere di **spostare gli appuntamenti** senza errori: la vecchia prenotazione verrà sovrascritta o eliminata, i messaggi rispecchieranno correttamente lo stato (nessun falso "posto occupato"), e gli orari proposti all'utente saranno esatti per la nuova data selezionata.

---

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 util.js
file://file_000000008fa0720a93b5e56a5ec20484